Chapter 4 — Lab 1: Build and register D365.Plugins

In this lab you build the D365.Plugins solution and register its 6 plugins and 9 steps in a Customer Service trial. All figures are real screenshots from the lab machine, taken on 26 Oct 2026.

Level	Beginner
Time	45 minutes
Source code	Plugins folder on GitHub
Tools	Visual Studio 2026 (2022 also works), XrmToolBox with the Plugin Registration tool
Environment	Dynamics 365 Customer Service trial, System Administrator role

Step 1 — Build the solution

1. Open `Plugins\D365.Plugins.sln` in Visual Studio. NuGet restores `Microsoft.CrmSdk.CoreAssemblies` automatically.
2. Set the configuration to **Release**.
3. Select **Build → Build Solution**.

Expected result: the Output window shows `Build: 1 succeeded, 0 failed` and the path of `D365.Plugins.dll` under `bin\Release\net48`.

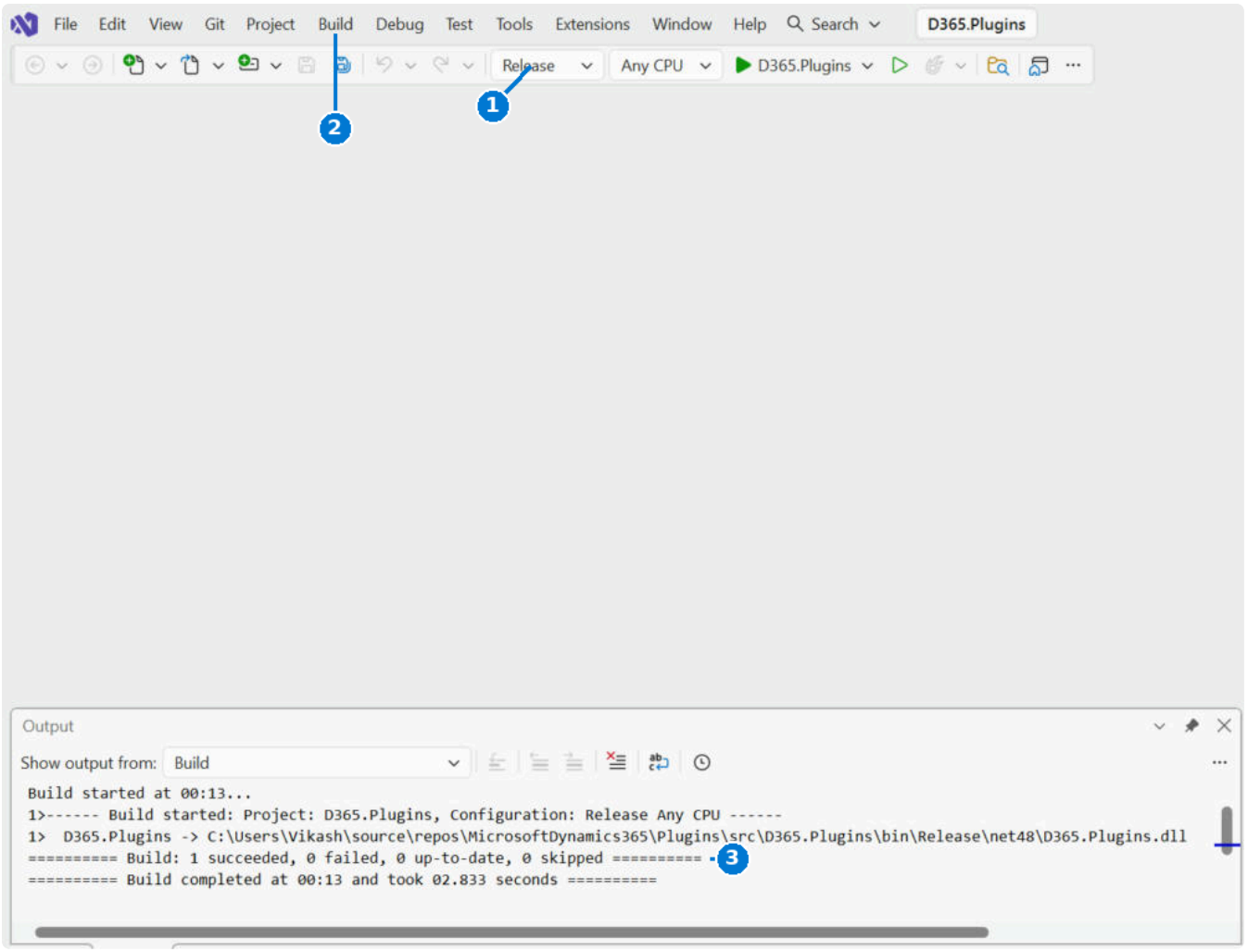


Figure 4.1 — Building D365.Plugins. (1) Release configuration. (2) Build menu → Build Solution. (3) Build: 1 succeeded, 0 failed.

Step 2 — Load the assembly

1. In XrmToolBox, connect to your environment and open **Plugin Registration**.
2. Select **Register → Register New Assembly**.
3. Paste the full path to `D365.Plugins.dll` in **Step #1** and select **Load Assembly**.
4. Keep **Sandbox** isolation and **Database** storage.

Expected result: the tree lists all six plugin classes, each marked *Isolatable*.

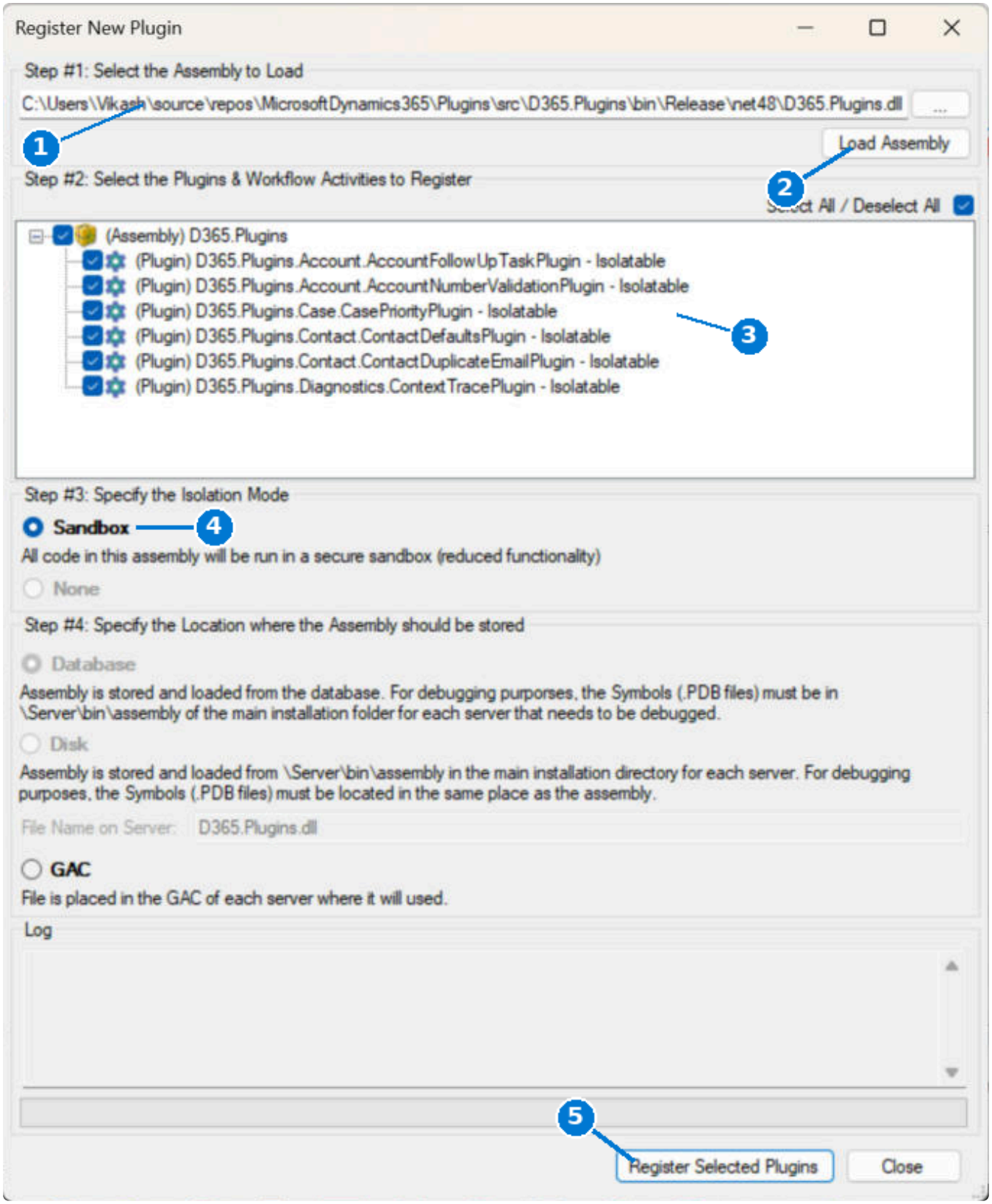


Figure 4.2 — Register New Plugin. (1) Path to D365.Plugins.dll. (2) Load Assembly. (3) The six plugin types found. (4) Sandbox isolation. (5) Register Selected Plugins.

Step 3 — Register the assembly

Select **Register Selected Plugins**.

Expected result: 1 assembly registered, 6 plugins registered, 0 errors.

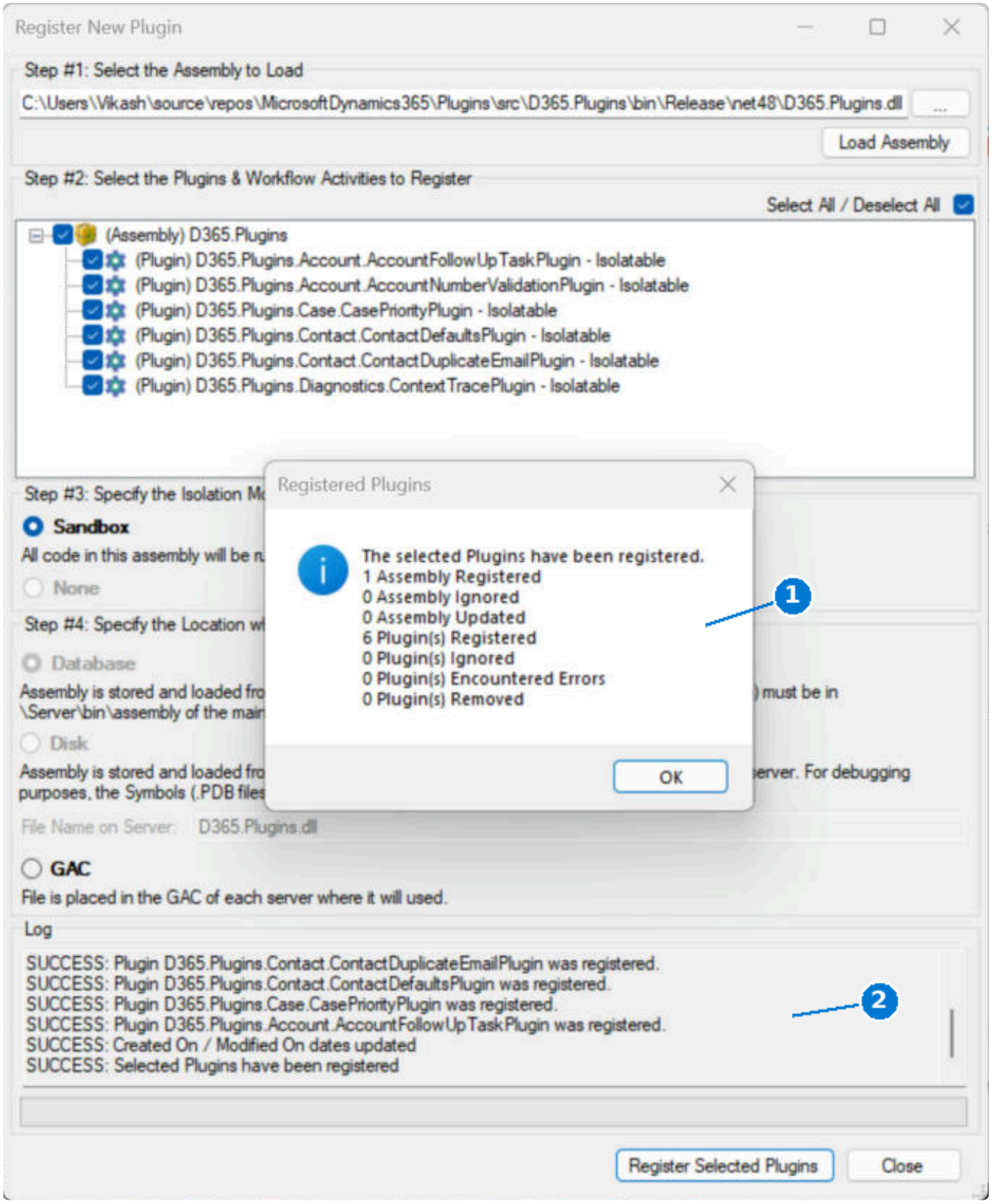


Figure 4.3 — Registration result. (1) 1 assembly and 6 plugins registered, 0 errors. (2) SUCCESS lines in the log.

Step 4 — Register the steps

For each row, select the plugin in the tree, then **Register → Register New Step** (Ctrl+T). Leave Execution Order at 1 and Run in User's Context as Calling User.

Plugin	Message	Table	Stage	Mode	Filtering attributes
AccountNumberValidationPlugin	Create	account	Pre-validation	Sync	—
AccountNumberValidationPlugin	Update	account	Pre-validation	Sync	accountnumber
AccountFollowUpTaskPlugin	Create	account	Post-operation	Sync	—
ContactDefaultsPlugin	Create	contact	Pre-operation	Sync	—
ContactDefaultsPlugin	Update	contact	Pre-operation	Sync	firstname, lastname
ContactDuplicateEmailPlugin	Create	contact	Pre-validation	Sync	—
ContactDuplicateEmailPlugin	Update	contact	Pre-validation	Sync	emailaddress1
CasePriorityPlugin	Create	incident	Pre-operation	Sync	—
ContextTracePlugin	Update	account	Post-operation	Async	name, accountnumber

To set filtering attributes, select **…** next to Filtering Attributes, search for the column, tick it, then **OK**.

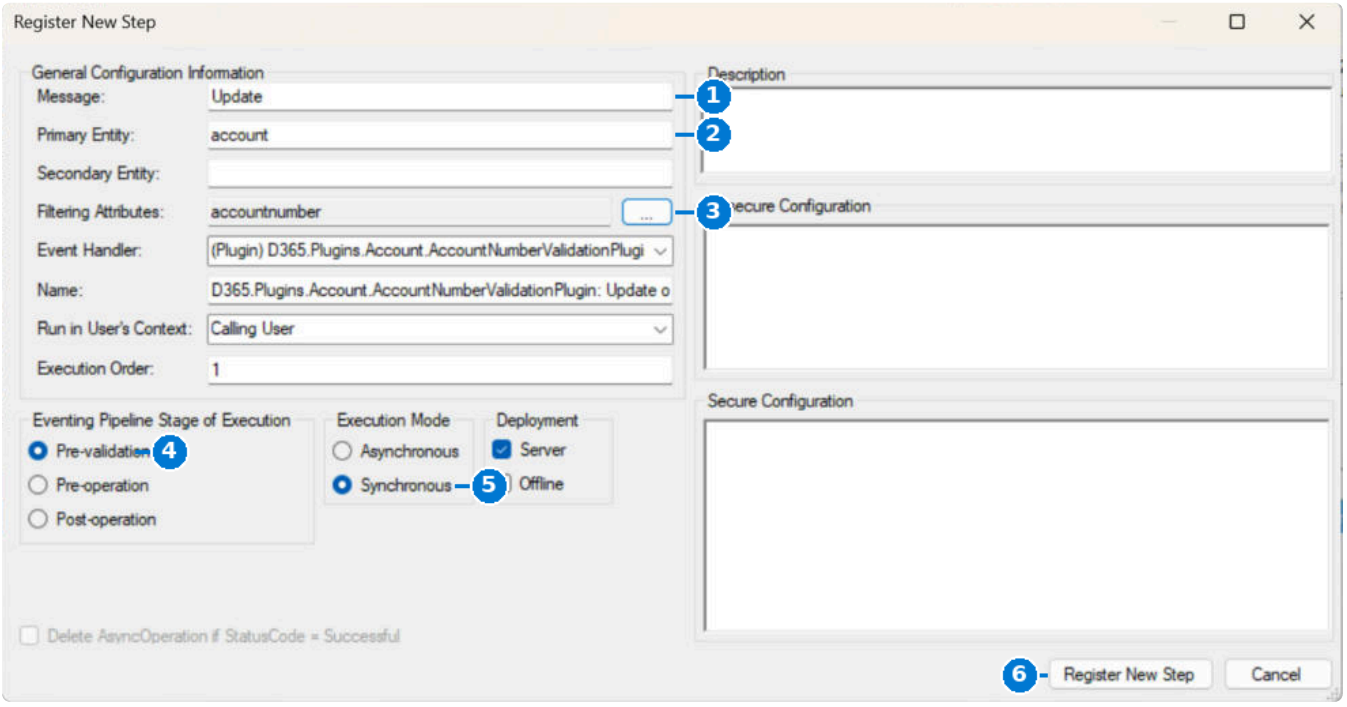


Figure 4.4 — Register New Step. (1) Message. (2) Primary Entity. (3) Filtering Attributes picker. (4) Pre-validation stage. (5) Synchronous mode. (6) Register New Step.

For **CasePriorityPlugin**, type `urgent,outage,down,critical` in **Unsecure Configuration**. The plugin reads this keyword list in its constructor.

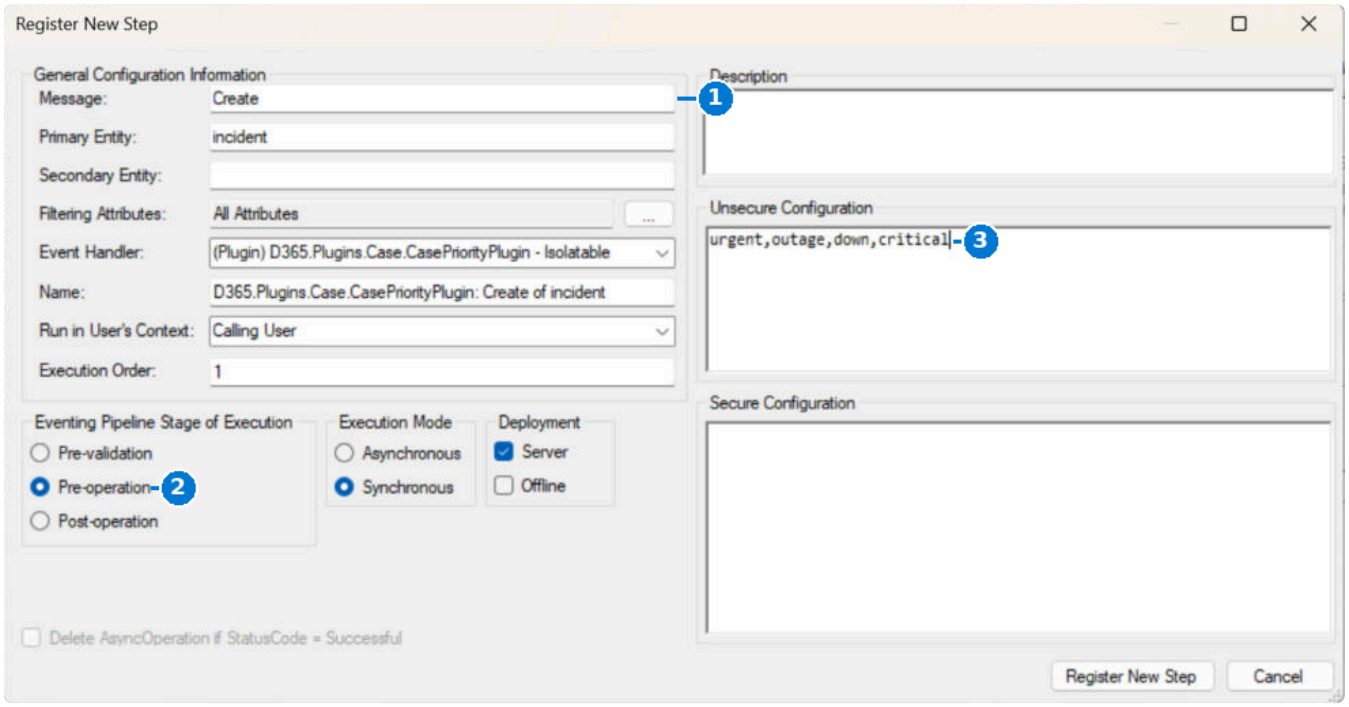


Figure 4.5 — Case Create step. (1) Message and table. (2) Pre-operation stage. (3) Keyword list in Unsecure Configuration.

> **Watch out:** an asynchronous Update step must have filtering attributes. Without them the tool shows "Filtering Attributes must be specified for this Message / Entity".

Step 5 — Add images to the trace step

1. Select the ContextTracePlugin step, then **Register → Register New Image** (Ctrl+I).
2. Tick **Pre Image**, set Name and Entity Alias to `PreImage`, and pick the columns `name` and `accountnumber`. Select **Register Image**.
3. Repeat with **Post Image** and `PostImage`.

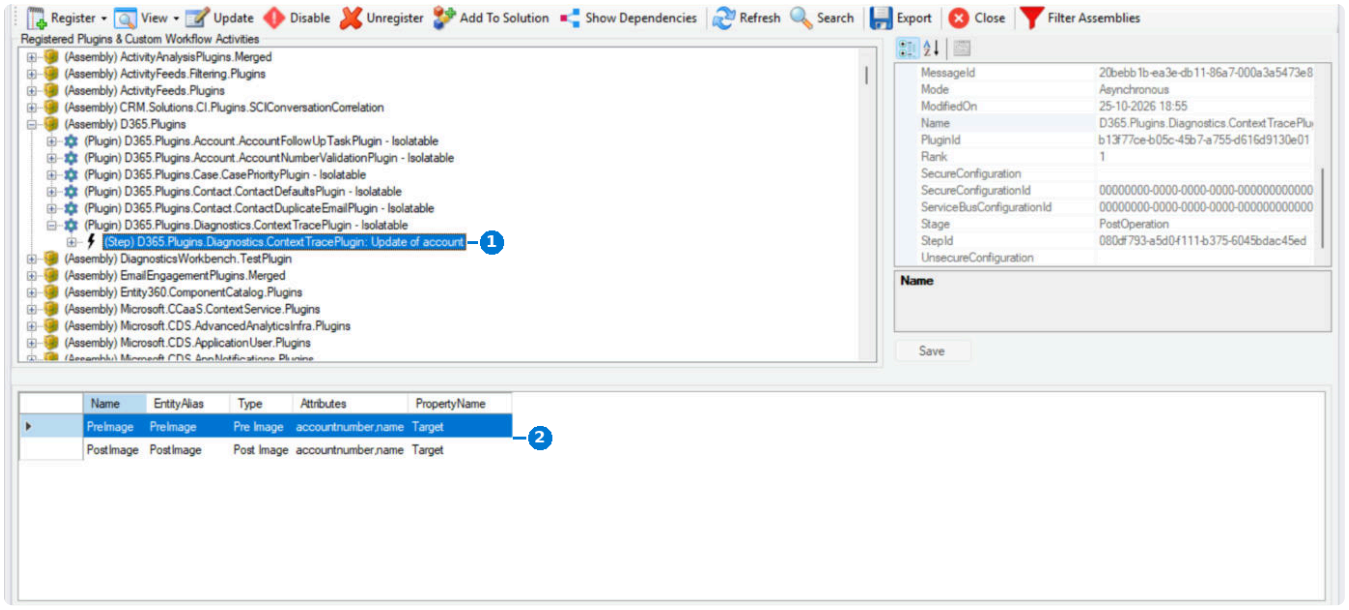


Figure 4.6 — Images on the trace step. (1) The ContextTracePlugin step. (2) PreImage and PostImage with name and accountnumber.

Step 6 — Check the result

Expand each plugin in the tree. You should see 9 steps under D365.Plugins.

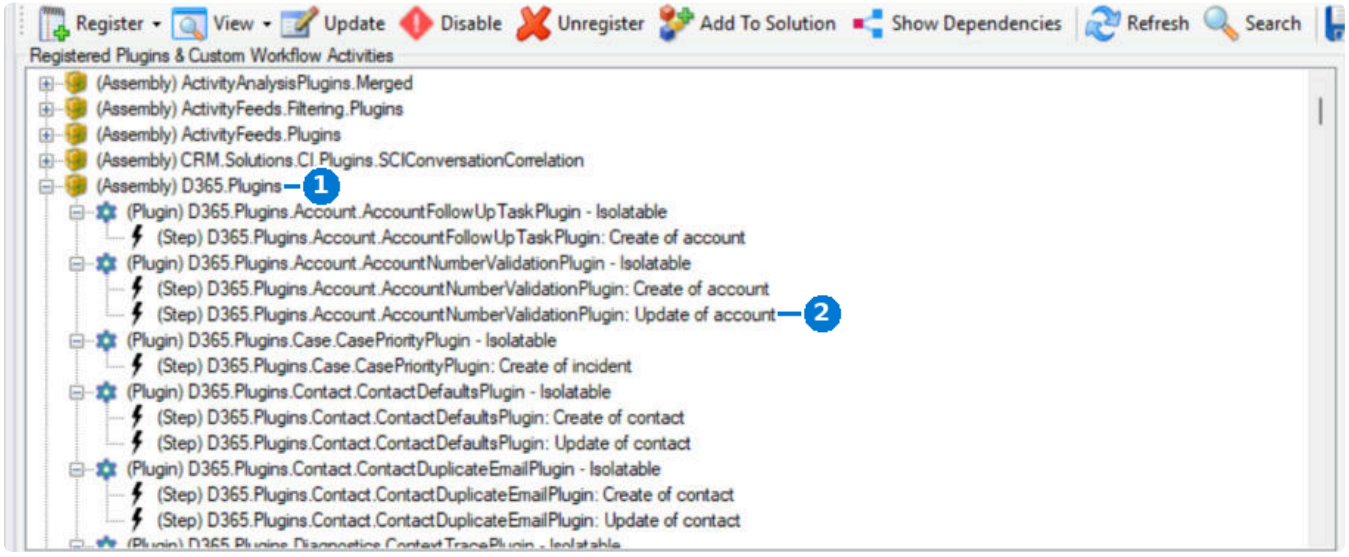


Figure 4.7 — Result. (1) The D365.Plugins assembly. (2) Steps listed under each plugin.

Step 7 — Test the plugins

These tests were run in the trial on 26 Oct 2026. Five of six plugins behaved as designed; one test failed and is explained below.

Test	Expected	Actual result	Pass?
Save a new account from the form (no Account Number field on it)	Blocked	"Account Number is required. Use the format ACC-12345." (Figure 4.8)	Pass
Create an account with Account Number `12345`	Blocked	"Account Number '12345' is not valid. Use the format ACC-12345 (ACC- followed by 5 digits)."	Pass
Create an account with Account Number `ACC-00001`	Saved, plus a follow-up task	Saved; task "Welcome call: Book Test - Contoso" due in 7 days (Figure 4.9)	Pass
Create a contact with First Name `John` and Last Name `Book Test`	Names trimmed	Saved as `John` and `Book Test`	Pass
Same contact has an email but no contact method	Preferred Method of Contact = Email	Stayed **Any**	**Fail**
Create a second contact with the same email	Blocked	"A contact with the email john.booktest@example.com already exists (John Book Test)…"	Pass
Create a case titled `Website outage in Mumbai office`	Priority = High	High	Pass
Create a case titled `Question about invoice`	Priority unchanged	Normal	Pass
Rename an account	Async trace log with both images	System Job succeeded; trace log shows PreImage and PostImage	Pass

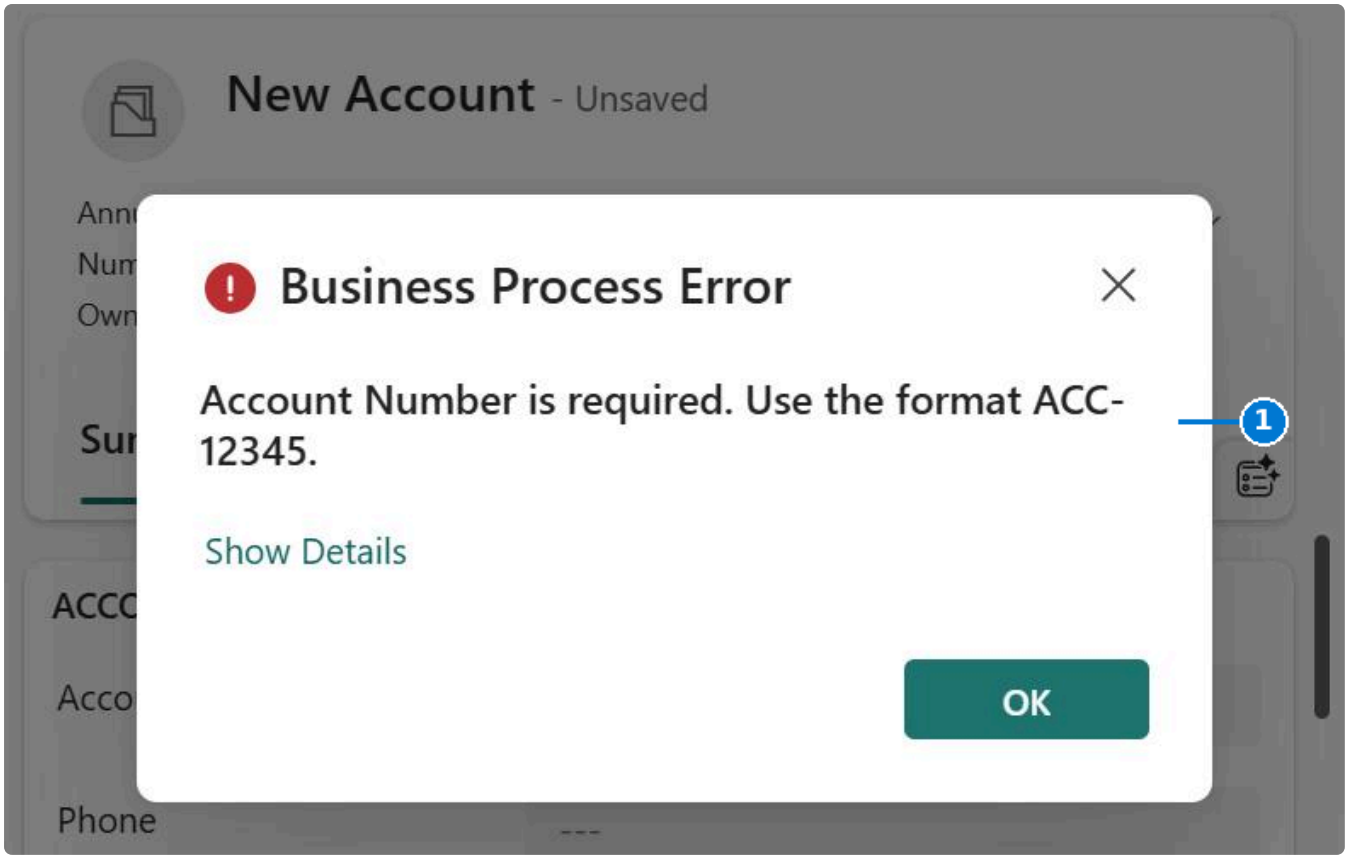


Figure 4.8 — The AccountNumberValidationPlugin blocks the save. (1) The message comes straight from InvalidPluginExecutionException.

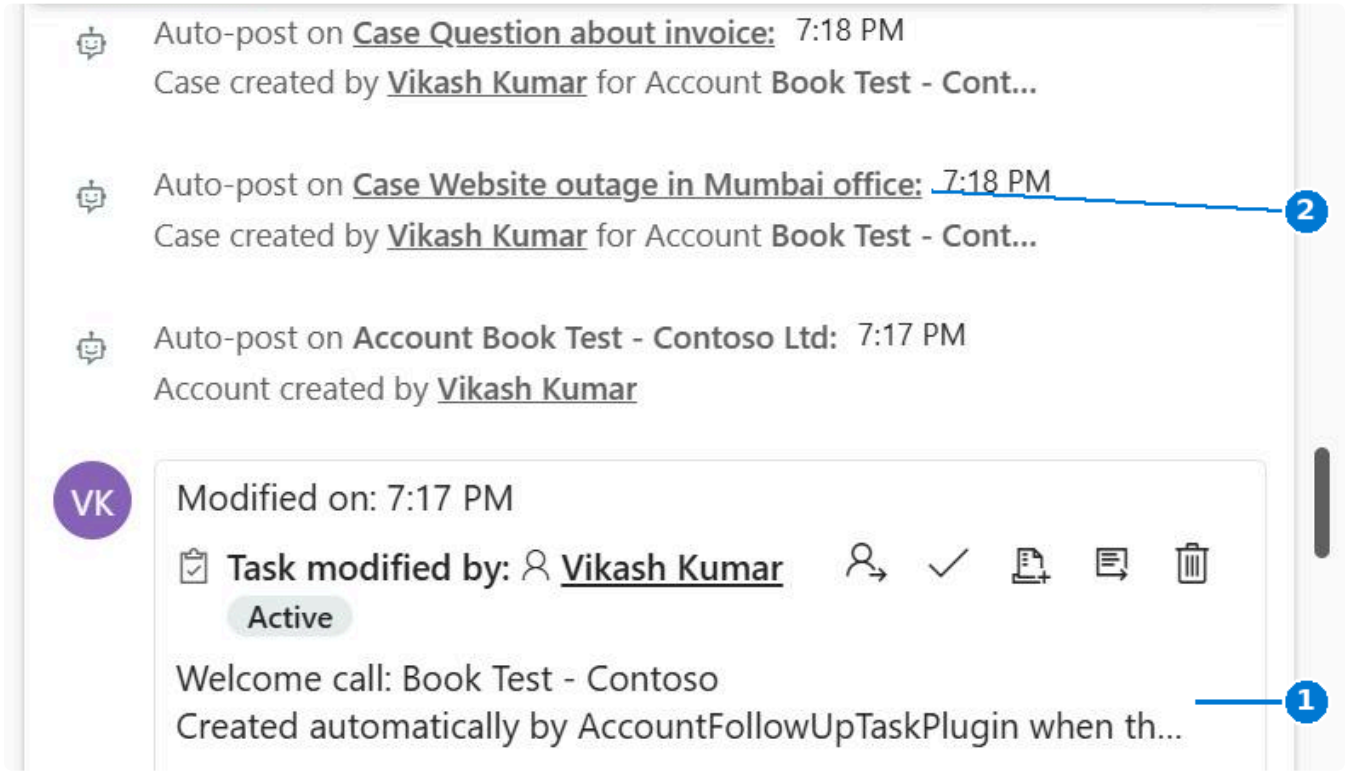


Figure 4.9 — Account timeline. (1) Task created by AccountFollowUpTaskPlugin. (2) The "outage" case, set to High by CasePriorityPlugin.

Trace log written by ContextTracePlugin after renaming the account (shortened):

```
MessageName: Update
Stage: 40
Mode: 1 (Asynchronous)
Depth: 1
IsInTransaction: False
--- InputParameters ---
Target: Entity account
    name = Book Test - Contoso Ltd
--- PreEntityImages ---
PreImage: name = Book Test - Contoso, accountnumber = ACC-00001
--- PostEntityImages ---
PostImage: name = Book Test - Contoso Ltd, accountnumber = ACC-00001
```

What the tests taught us

- **Match plugin rules to the form.** The Copilot Service workspace account form has no Account Number field, so with this plugin users can't create accounts from it at all. Either add the field to the form or relax the rule on Create. A plugin runs everywhere, so check every app and integration that writes the table.
- **Default values arrive before PreOperation.** The contact test failed because Dataverse fills Preferred Method of Contact with its default (Any = 1) before stage 20. The plugin only checked for an empty value. Fix: treat `1` (Any) as "not chosen" too. This fix is covered in Chapter 7.
- **Changing name parts doesn't rebuild Full Name.** The duplicate message shows `John Book Test`: Full Name kept the original spacing even though Last Name was cleaned. Clean name parts in PreValidation instead, or set Full Name yourself. Chapter 7 compares both.
- **Async steps report through System Jobs.** IsInTransaction is False and the result is a System Job (status Succeeded), not a message to the user.

Common errors

Symptom	Cause	Fix
Load Assembly does nothing	The path box didn't register the change	Press Tab after pasting, then select Load Assembly
"Filtering Attributes must be specified"	Async Update step with no filtering attributes	Pick the columns the step depends on

Symptom	Cause	Fix
Plugin doesn't fire on Update	The changed column isn't in the filtering attributes	Add the column, or test by changing a filtered column
Error mentions strong name	Assembly not signed	Keep `SignAssembly` and `D365.Plugins.snk` in the project

Chapter 5 — The execution context and entity images

Everything your plugin knows about the current operation comes from one object: the **execution context**. This chapter shows what's in it, how the D365.Plugins base class reads it, and how entity images give you the record before and after a change.

In this chapter you will learn to:

- get the context, organization service and tracing service from `IServiceProvider`
- read Target, images, depth and users safely
- know which images exist for each message and stage

5.1 Services the pipeline gives you

Service	Get it with	Use it to
`IPluginExecutionContext`	`serviceProvider.GetService(typeof(IPluginExecutionContext))`	Read the message, table, stage, Target, images, depth, users
`IOrganizationServiceFactory`	`serviceProvider.GetService(typeof(IOrganizationServiceFactory))`	Create an `IOrganizationService` to read and write data
`ITracingService`	`serviceProvider.GetService(typeof(ITracingService))`	Write lines to the Plug-in Trace Log

`LocalPluginContext` in D365.Plugins does this once for every plugin:

```
PluginExecutionContext =
(IPluginExecutionContext)serviceProvider.GetService(typeof(IPluginExecutionContext));
TracingService =
(ITracingService)serviceProvider.GetService(typeof(ITracingService));
ServiceFactory =
(IOrganizationServiceFactory)serviceProvider.GetService(typeof(IOrganizationServiceFactory));

// Runs as the user the step is configured to run as.
OrganizationService =
ServiceFactory.CreateOrganizationService(PluginExecutionContext.UserId);
```

It also offers `SystemOrganizationService`, created with `CreateOrganizationService(null)`, which runs as the SYSTEM user. Use it only for checks that must see every record, such as duplicate detection in Chapter 6.

5.2 Key context properties

Property	What it tells you
`MessageName`	"Create", "Update", "Delete"…
`PrimaryEntityName`	Table logical name, e.g. `account`
`PrimaryEntityId`	The record's ID (empty in PreValidation/PreOperation of Create unless the caller supplied it)
`Stage`	10, 20, 30 or 40
`Mode`	0 = synchronous, 1 = asynchronous
`Depth`	1 for the original request; +1 per nested call
`InputParameters["Target"]`	`Entity` on Create/Update (only changed columns on Update); `EntityReference` on Delete
`OutputParameters["id"]`	New record ID on Create, PostOperation
`PreEntityImages` / `PostEntityImages`	Snapshots you registered on the step
`SharedVariables`	Values passed between steps of the same operation
`UserId` / `InitiatingUserId`	Who the step runs as / who started the request
`ParentContext`	The context of the operation that triggered this one
`IsInTransaction`	True inside the database transaction
`CorrelationId`	One ID across every plugin in the same request

Read Target safely. On Update, Target holds only the columns being changed. Check before you read:

```
var target = localContext.GetTargetEntity();      // null when Target isn't
an Entity
if (target == null || !target.Contains("accountnumber")) return;
var number = target.GetAttributeValue<string>("accountnumber");
```

5.3 Entity images

An image is a copy of the record's columns that Dataverse passes to your step. A **pre image** is the record before the change; a **post image** is the record after it. You get only the images and columns you register.

Message	PreValidation	PreOperation	PostOperation
Create	—	—	Post
Update	Pre	Pre	Pre and Post
Delete	Pre	Pre	Pre

Register only the columns you need. Use a pre image instead of retrieving the record again: it's faster and saves a service call.

Lab 2 — Trace the execution context

Level: Beginner · **Time:** 20 minutes · **Uses:** `ContextTracePlugin`, already registered in Lab 1 on Account Update, PostOperation, asynchronous, with PreImage and PostImage (name, accountnumber).

1. Turn on tracing: **Settings (gear) → Advanced settings → Settings → Administration → System Settings → Customization → Enable logging to plug-in trace log = All**.
2. Open any account and change its **Account Name**. Save.
3. Wait about 30 seconds (the step is asynchronous).
4. Open the trace log: in Advanced settings go to **Settings → Plug-in Trace Log**, or browse to `/main.aspx?pagetype=entitylist&etn=plugintracelog`. Open the newest row for `ContextTracePlugin`.

Expected result — real output from the trial, shortened:

```
Entered ContextTracePlugin. Message: Update, Table: account, Stage: 40, Mode:
1, Depth: 1
=== Execution context ===
Mode: 1 (Asynchronous)
Depth: 1
IsInTransaction: False
HasParentContext: True
--- InputParameters (2) ---
Target: Entity account
    name = String Book Test - Contoso Ltd
    modifiedon = DateTime 10/25/2026 7:18:24 PM
ConcurrencyBehavior: ConcurrencyBehavior Default
--- SharedVariables (2) ---
IsAutoTransact: Boolean True
AcceptLang: String en-US
--- PreEntityImages (1) ---
PreImage: accountnumber = ACC-00001, name = Book Test - Contoso
--- PostEntityImages ---
PostImage: accountnumber = ACC-00001, name = Book Test - Contoso Ltd
```

What to notice:

- Target contains only `name` (and system columns), not `accountnumber`. The images give you both.
- PreImage has the old name; PostImage has the new one. Comparing them tells you exactly what changed.
- The platform adds its own SharedVariables, such as `IsAutoTransact`.

Turn tracing back to Exception or Off when you finish; trace logs use storage.

Common errors

Symptom	Cause	Fix
`KeyNotFoundException` for an image	Image not registered, or alias spelled differently	Check the alias on the step; use `TryGetValue`
Column missing from Target on Update	It didn't change	Read it from a pre image
No trace log rows	Tracing is Off, or the async job hasn't run yet	Set tracing to All; check System Jobs

Knowledge check

1. On Update, why might `target.Contains("name")` be false even though the record has a name?
2. In which stage can you have both a pre image and a post image?
3. What's the difference between `UserId` and `InitiatingUserId`?

Answers: 1. Target holds only changed columns. 2. PostOperation of Update. 3. UserId is who the step runs as (it can be set to a specific user); InitiatingUserId is who started the request.

Summary

The execution context tells you what is happening; Target holds the change; images hold the record before and after. Read everything defensively, register only the columns you need, and use the trace log to see exactly what your plugin received.

Chapter 6 — Validation plugins

A validation plugin checks a request and stops it with a clear message when the data breaks a rule. Because it runs on the server, the rule holds for forms, imports, flows and the API alike.

In this chapter you will learn to:

- reject a save with `InvalidPluginExecutionException`
- validate a single field and check for duplicates across records
- choose the stage and filtering attributes for a validation step

6.1 How to stop a save

Throw `InvalidPluginExecutionException`. Dataverse cancels the operation, rolls back the transaction and shows your message to the user (Figure 6.1). Any other exception type also stops the save, but the user sees a generic error, so always wrap expected failures in `InvalidPluginExecutionException` with a message that says what to fix.

Good message	Poor message
Account Number '12345' is not valid. Use the format ACC-12345.	Validation failed.
A contact with the email jo@example.com already exists (Jo Smith).	Duplicate record.

6.2 Which stage?

Use **PreValidation (10)** for checks that only need the request: it runs first and can stop the operation before any other work. Use **PreOperation (20)** if your check must run inside the transaction, for example because it reads records changed earlier in the same save.

On Update, always set **filtering attributes** to the columns you validate, so the plugin doesn't run on unrelated changes.

6.3 Plugin 1 — AccountNumberValidationPlugin

Rule: an account must have an Account Number in the format `ACC-12345`.

```
public class AccountNumberValidationPlugin : PluginBase
{
    private const string AccountNumberAttribute = "accountnumber";
    private static readonly Regex AccountNumberPattern = new Regex(@"^ACC-
\d{5}$", RegexOptions.Compiled);

    protected override void ExecutePlugin(LocalPluginContext localContext)
    {
        var context = localContext.PluginExecutionContext;
        var target = localContext.GetTargetEntity();
        if (target == null) return;

        var isCreate = context.MessageName == "Create";

        // On Update, Target only holds changed columns.
        if (!isCreate && !target.Contains(AccountNumberAttribute)) return;

        var accountNumber = target.GetAttributeValue<string>
(AccountNumberAttribute);
        if (string.IsNullOrWhiteSpace(accountNumber))
            throw new InvalidPluginExecutionException("Account Number is
required. Use the format ACC-12345.");

        accountNumber = accountNumber.Trim();
        if (!AccountNumberPattern.IsMatch(accountNumber))
            throw new InvalidPluginExecutionException(string.Format(
                "Account Number '{0}' is not valid. Use the format ACC-12345
(ACC- followed by 5 digits).", accountNumber));
    }
}
```

Key lines: the `Regex` is `static readonly`, so it's built once and is safe to share across threads. On Update the plugin returns early when Account Number isn't in Target, because the filtering attribute already limits when it runs.

Registration: Create and Update of `account`, PreValidation, synchronous; Update filtered on `accountnumber`.

6.4 Plugin 2 — ContactDuplicateEmailPlugin

Rule: no two active contacts may share the same primary email.

```
var query = new QueryExpression("contact")
{
    ColumnSet = new ColumnSet("fullname"),
    TopCount = 1,
    NoLock = true
};
query.Criteria.AddCondition("emailaddress1", ConditionOperator.Equal, email);
query.Criteria.AddCondition("statecode", ConditionOperator.Equal, 0);

// On Update, ignore the record being saved.
if (context.MessageName == "Update" && target.Id != Guid.Empty)
    query.Criteria.AddCondition("contactid", ConditionOperator.NotEqual,
target.Id);

var matches = localContext.SystemOrganizationService.RetrieveMultiple(query);
if (matches.Entities.Count > 0)
    throw new InvalidPluginExecutionException(string.Format(
        "A contact with the email {0} already exists ({1}). Open that contact
instead of creating a duplicate.",
        email, matches.Entities[0].GetAttributeValue<string>("fullname")));
```

Key lines: the query asks for one column and one row (`TopCount = 1`), which is all it needs. It runs as SYSTEM so it finds duplicates the user can't see. On Update, the current contact is excluded.

Limit: two saves at the same instant can both pass the check. For a hard guarantee, also add an **alternate key** on `emailaddress1`; Dataverse then rejects the second insert at the database level.

Registration: Create and Update of `contact`, PreValidation, synchronous; Update filtered on `emailaddress1`.

Lab 3 — Test the validation plugins

Level: Beginner · **Time:** 25 minutes · **Uses:** plugins registered in Lab 1.

1. Open **Accounts → New**, enter an Account Name and save. In the Copilot Service workspace app the form has no Account Number field, so the plugin blocks the save (Figure 6.1).
2. Create a contact with an email address, for example `john.booktest@example.com`, and save.
3. Create a second contact with the same email and save (Figure 6.2).

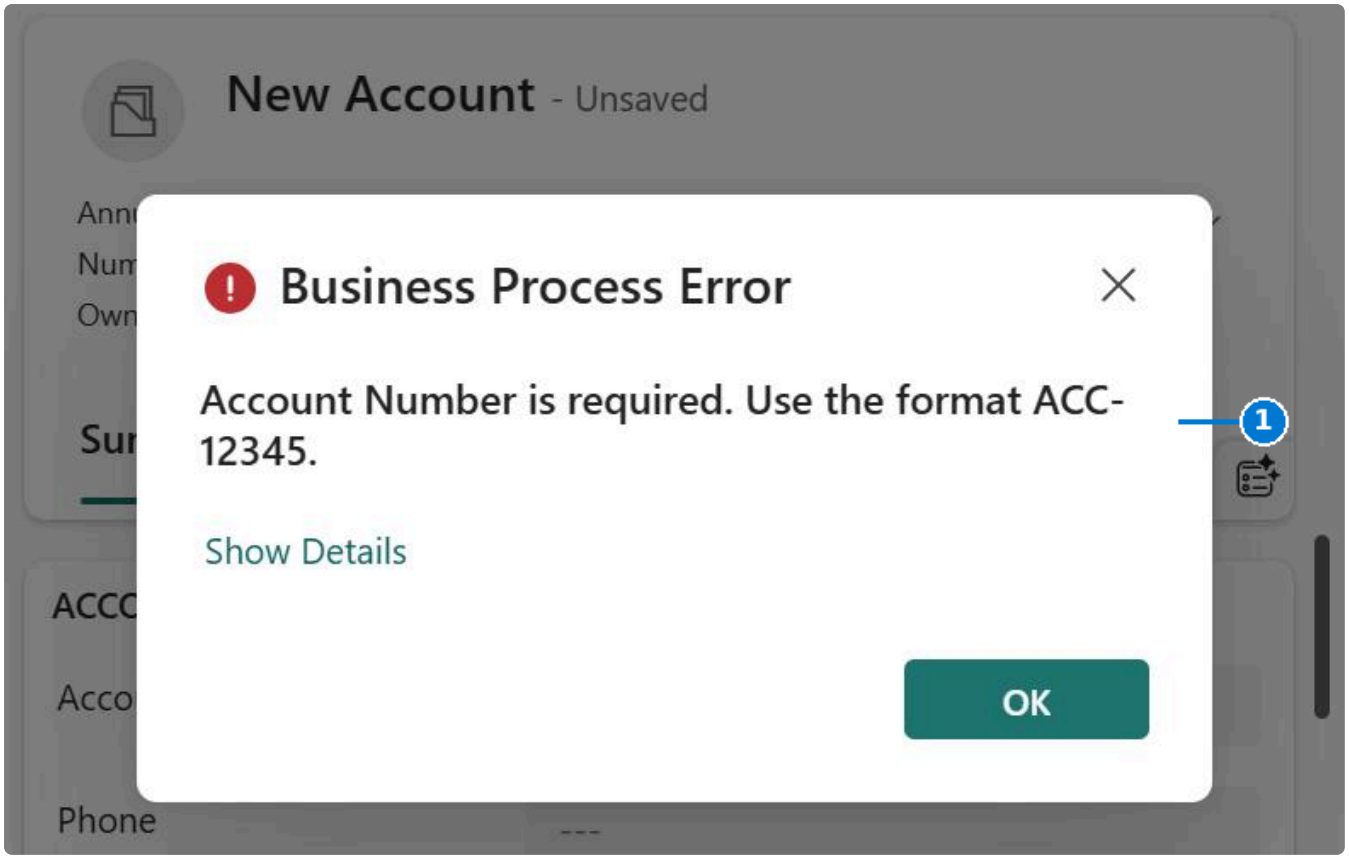


Figure 6.1 — AccountNumberValidationPlugin blocks the save. (1) The text comes from InvalidPluginExecutionException.

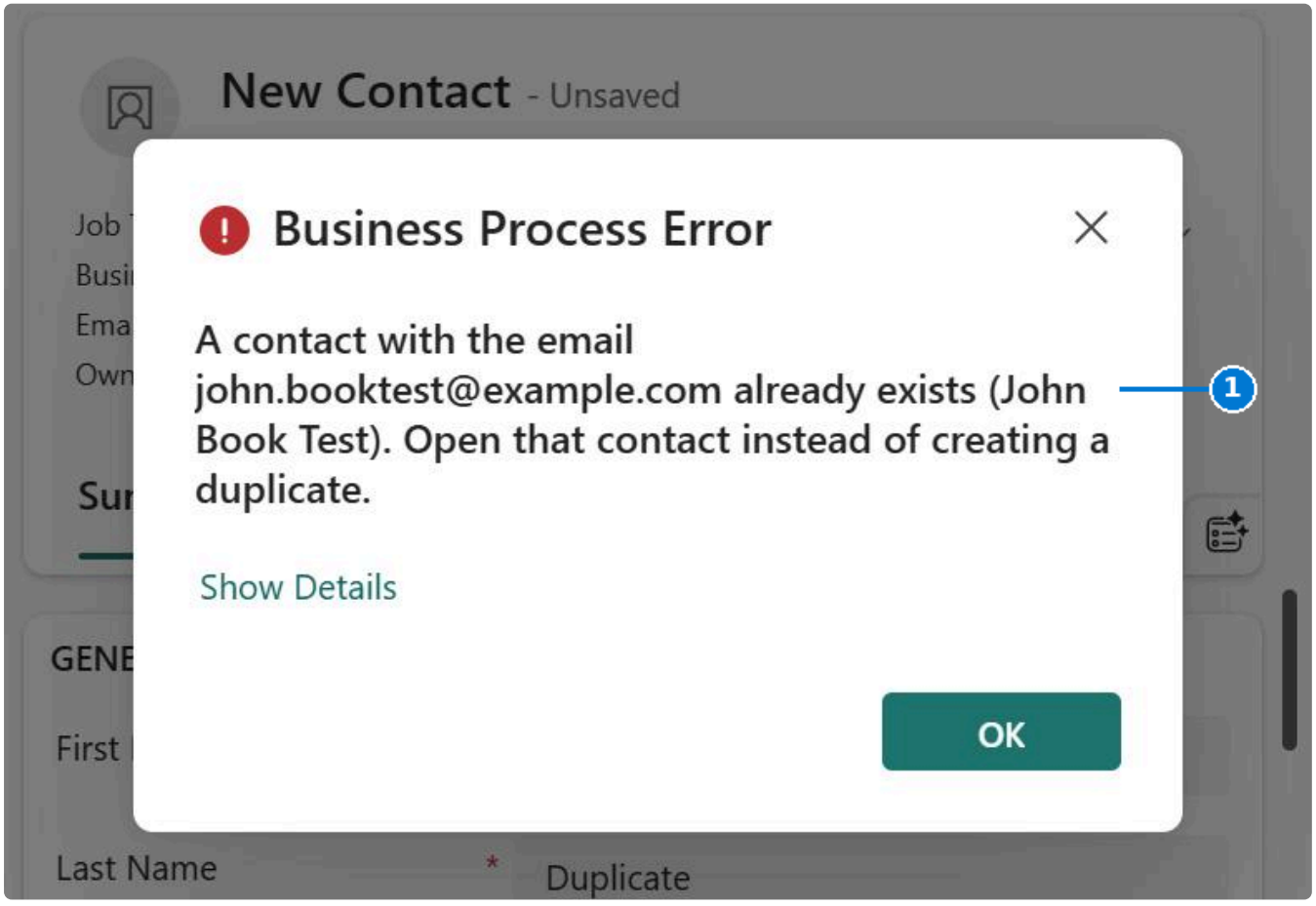


Figure 6.2 — ContactDuplicateEmailPlugin names the existing contact so the user can open it. (1) Your message, with the duplicate's name.

Results from the trial (26 Oct 2026):

Test	Result
Account saved from the form with no number	Blocked: "Account Number is required…"
Account created through the API with `12345`	Blocked: "Account Number '12345' is not valid…"
Account created with `ACC-00001`	Saved
Second contact with the same email (form and API)	Blocked, naming the existing contact

Lesson from the test: the account rule makes accounts impossible to create from a form that doesn't show Account Number. Before you enforce a required field in a plugin, add it to every form that creates the record, or apply the rule only on Update.

Common errors

Symptom	Cause	Fix
Users see "An unexpected error occurred"	A non-InvalidPluginExecutionException was thrown	Catch expected cases and throw InvalidPluginExecutionException
Duplicate check misses records	Query runs as the user, who can't see them	Run the check with the SYSTEM service
Validation runs on every update	No filtering attributes	Filter on the validated columns

Knowledge check

1. Why use PreValidation for these checks?
2. Why does the duplicate check use `TopCount = 1`?
3. What closes the gap when two duplicates are saved at the same moment?

Answers: 1. It runs first and only needs the request, so bad data is stopped before other work. 2. One match is enough to block; fewer rows means a faster query. 3. An alternate key on the email column.

Summary

Validate in PreValidation, throw InvalidPluginExecutionException with a message that tells the user what to fix, filter Update steps on the validated columns, and back duplicate checks with an alternate key.

Chapter 7 — Changing data: defaults, related records, configuration

Most plugins don't just say no; they change data. This chapter covers the three patterns you'll use most: set values on the record before it's saved, create a related record after it's saved, and make behaviour configurable without recompiling.

In this chapter you will learn to:

- change Target in PreOperation so values save in the same write
- create related records in PostOperation
- read unsecure and secure configuration from a step
- find and fix a real defaulting bug

7.1 Pattern 1 — Set values in PreOperation

In PreOperation, anything you put on Target is saved with the record. There is no extra Update call, so no extra trigger and no loop risk.

`ContactDefaultsPlugin` trims names and defaults the contact method:

```
CleanName(localContext, target, "firstname");
CleanName(localContext, target, "lastname");

if (context.MessageName == "Create"
    && target.GetAttributeValue<OptionSetValue>("preferredcontactmethodcode")
== null
    && !string.IsNullOrWhiteSpace(target.GetAttributeValue<string>
("emailaddress1")))
{
    target["preferredcontactmethodcode"] = new OptionSetValue(2); // Email
}
```

Registration: contact Create and Update, PreOperation, synchronous; Update filtered on `firstname, lastname`.

What the test showed

We created a contact with First Name `John` , Last Name `Book Test` and an email.

Check	Expected	Actual
First and last name trimmed	`John` , `Book Test`	`John` , `Book Test` — pass

Check	Expected	Actual
Preferred Method of Contact	Email (2)	**Any (1) — fail**
Full Name	`John Book Test`	`John Book Test` (extra spaces kept)

Why the method stayed "Any": Dataverse applies the column's default value (Any = 1) to the request before PreOperation. By the time the plugin runs, the value isn't null, so the condition is false.

The fix treats Any as "not chosen":

```
const int PreferredContactMethodAny = 1;
const int PreferredContactMethodEmail = 2;

var method = target.GetAttributeValue<OptionSetValue>
("preferredcontactmethodcode");
if (context.MessageName == "Create"
    && (method == null || method.Value == PreferredContactMethodAny)
    && !string.IsNullOrWhiteSpace(target.GetAttributeValue<string>
("emailaddress1")))
{
    target["preferredcontactmethodcode"] = new
OptionSetValue(PreferredContactMethodEmail);
}
```

Trade-off: a user who deliberately picks Any on create will also get Email. If that matters, set the column's default to blank instead and keep the original null check.

Why Full Name kept the spaces: In the trial, Full Name was built from the original name parts, not the values the plugin cleaned in PreOperation. To get a clean Full Name, register the name-cleaning step in **PreValidation** instead, so the cleaned values are in the request from the start. *This change and the fix above are in the book's code but haven't yet been redeployed and retested in the trial; test them before relying on them.*

7.2 Pattern 2 — Create related records in PostOperation

In PostOperation the new record has an ID, so you can link other records to it. `AccountFollowUpTaskPlugin`:

```
var accountId = context.PrimaryEntityId;

var task = new Entity("task");
task["subject"] = string.Format("Welcome call: {0}", accountName);
task["regardingobjectid"] = new EntityReference("account", accountId);
task["scheduledend"] = DateTime.UtcNow.AddDays(7);

localContext.OrganizationService.Create(task);
```

Because the step is synchronous, the task and the account are in one transaction: if the task can't be created, the account isn't saved either. Make the step asynchronous only if the account should save even when the task fails.

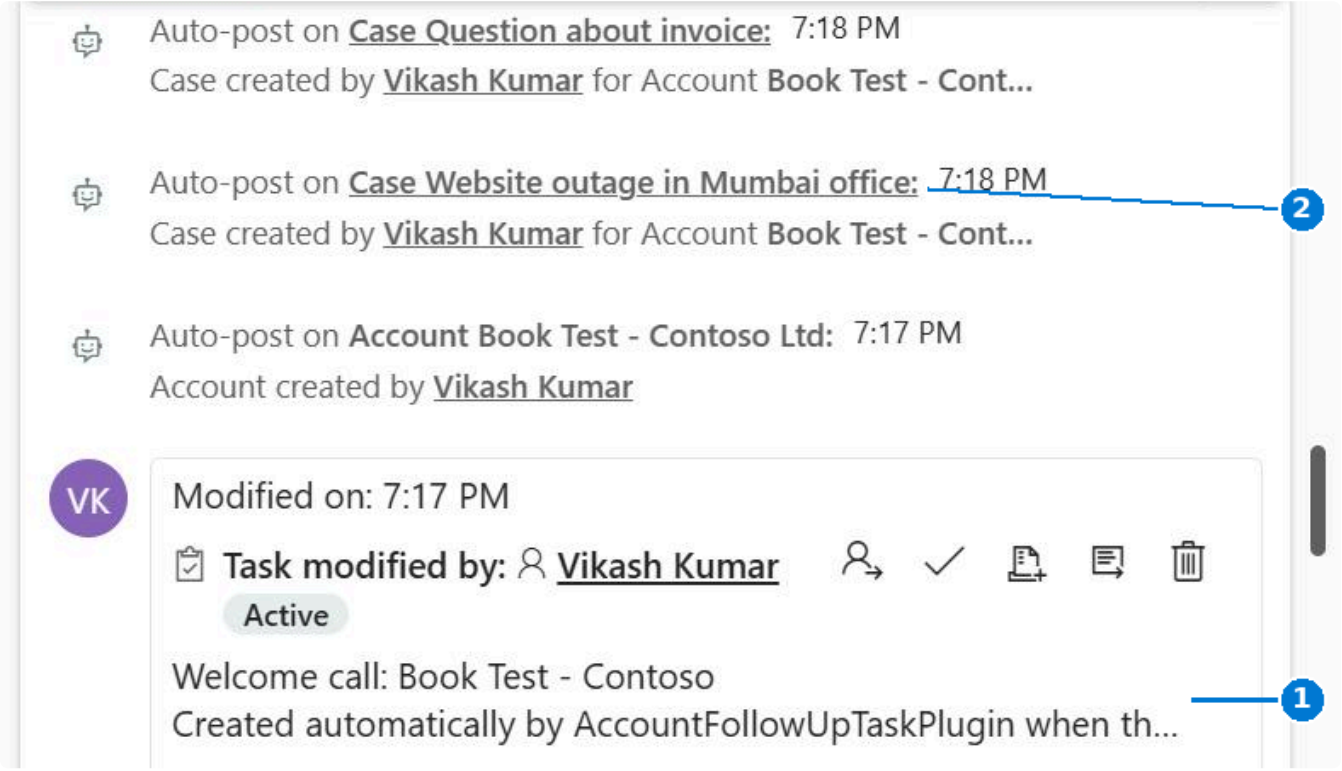


Figure 7.1 — Account timeline in the trial. (1) The task created by AccountFollowUpTaskPlugin. (2) A case set to High by CasePriorityPlugin.

7.3 Pattern 3 — Make it configurable

Every step has two configuration boxes. Dataverse passes them to your plugin's constructor:

```
public CasePriorityPlugin(string unsecureConfiguration, string
secureConfiguration)
    : base(unsecureConfiguration, secureConfiguration)
{
    // Parsed once per instance and never changed: safe across threads.
    _keywords = ParseKeywords(unsecureConfiguration);   //
"urgent,outage,down,critical"
}
```

	Unsecure configuration	Secure configuration
Who can read it	Anyone who can read the step	System Administrators only
Moves with a solution	Yes	No; set it in each environment
Use for	Keywords, thresholds, switches	Values that must not be visible to customizers

Neither box is a secrets vault. Keep real secrets such as API keys in **Azure Key Vault** and reference them with an environment variable.

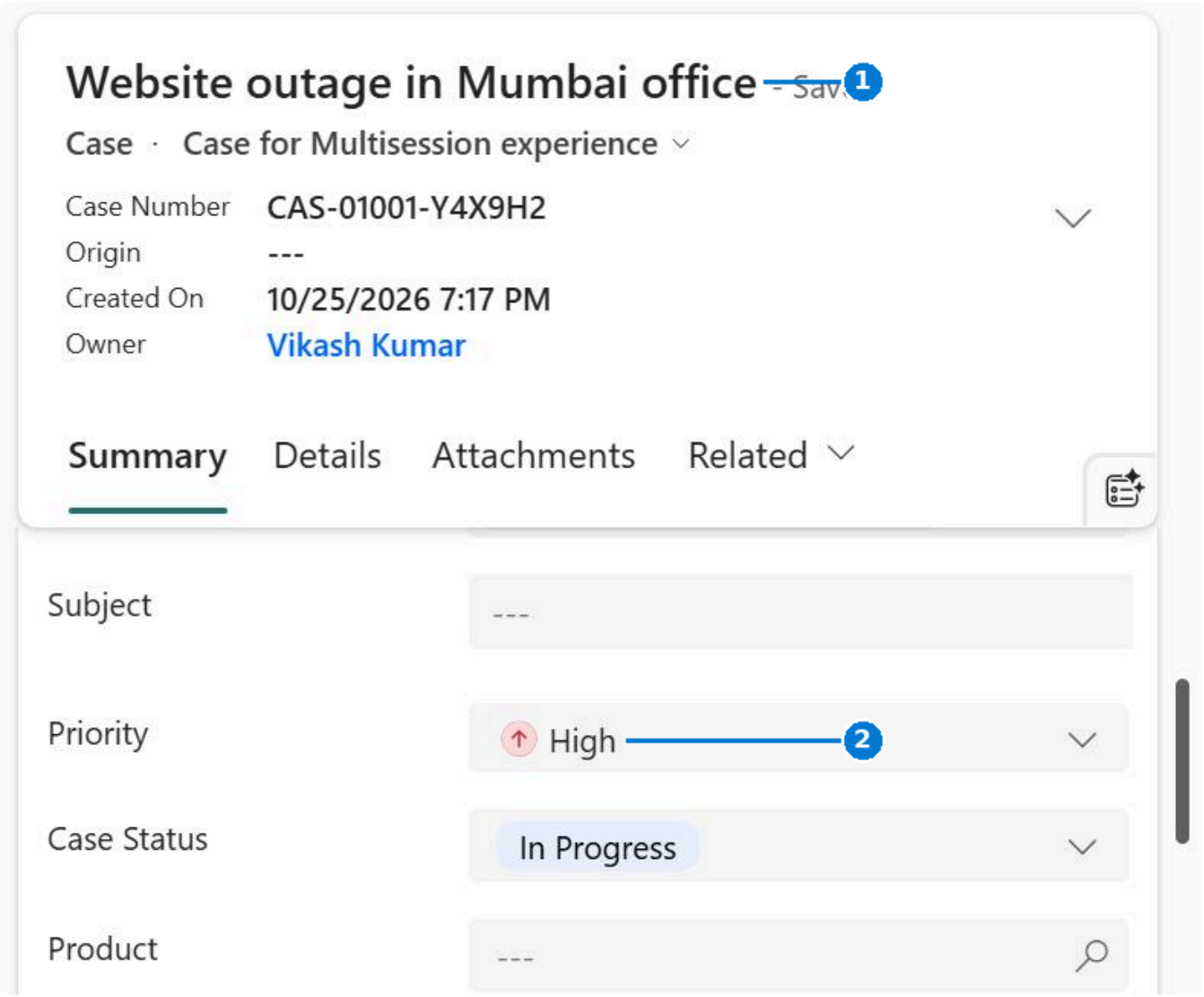


Figure 7.2 — Case "Website outage in Mumbai office" in the trial. (1) The title contains a configured keyword. (2) Priority was set to High by the plugin.

Lab 4 — Defaults, related records and configuration

Level: Intermediate · **Time:** 30 minutes · **Uses:** plugins registered in Lab 1.

1. Create a contact with extra spaces in the names and an email. Open it and check the names and Preferred Method of Contact.
2. Create an account with Account Number `ACC-00002` (through a form that shows the field, or the Web API). Open its timeline.
3. Create two cases for that account: `Website outage in Mumbai office` and `Question about invoice`. Compare their priorities.
4. In the registration tool, change the CasePriorityPlugin step's Unsecure Configuration to `urgent,vip` and create a case titled `VIP customer call`.

Expected results (steps 1–3 checked in the trial): names trimmed; Preferred Method of Contact stays Any until you deploy the fix; a "Welcome call" task due in 7 days; the outage case is High and the invoice case Normal. Step 4 should give High, because `vip` is now a keyword.

Knowledge check

1. Why set values in PreOperation rather than calling Update in PostOperation?
2. Why did the default value check fail in the trial?
3. Where should you keep an API key a plugin needs?

Answers: 1. It saves in the same write, with no extra trigger or loop risk. 2. Dataverse applied the column default (Any) before PreOperation, so the value wasn't null. 3. Azure Key Vault, referenced through an environment variable, not in step configuration.

Summary

Change the record in PreOperation, create related records in PostOperation, and put tunable values in step configuration. Always test against real platform behaviour: defaults and calculated columns may already be set when your plugin runs.

Chapter 8 — Avoiding loops and duplicates

The two most common production bugs in plugins are loops (a plugin triggers itself) and duplicates (a plugin creates the same record twice). Both come from the same root: a plugin that doesn't check whether its work is already done.

In this chapter you will learn to:

- explain how recursion happens and what Depth tells you
- prevent loops with filtering attributes, value checks and stage choice
- explain why checking `Depth > 1` alone isn't enough
- make record creation idempotent

8.1 How a loop happens

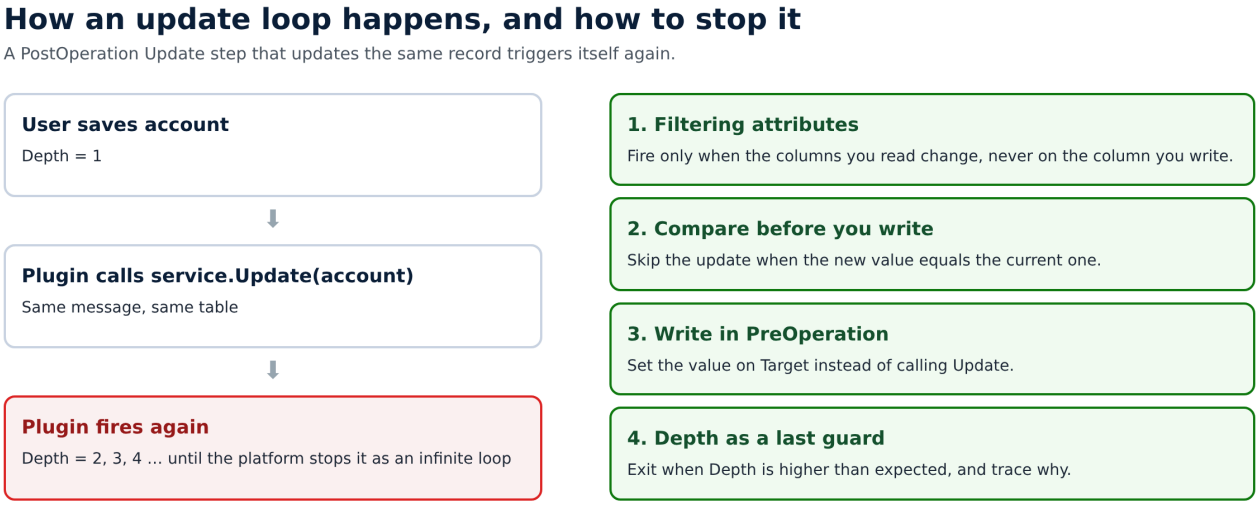


Figure 8.1 — A PostOperation Update step that updates the same record fires itself again. Use the four guards on the right, in that order.

Each nested call raises `context.Depth` by one. By default Dataverse stops a chain that goes deeper than 8 within an hour and returns an infinite-loop error, so a loop shows up as a failed save rather than a hang. You still want to prevent it: every pass through the loop costs time, and the save fails.

8.2 Four guards, in order

Guard	How	Stops
1. Filtering attributes	Register Update steps only on the columns you read	Runs triggered by your own write to other columns
2. Compare before you write	Skip the update if the value is already correct	Repeated writes of the same value
3. Write in PreOperation	Put the value on Target instead of calling Update	The extra Update and its trigger entirely
4. Depth guard	`if (context.Depth > expected) return;` with a trace line	Unexpected chains you didn't foresee

Why `Depth > 1` alone isn't enough

Depth goes up for every nested call, not just your own loop. If a Power Automate flow or another plugin updates the account, your step runs with Depth 2 and a blanket `Depth > 1` check silently skips it, so your rule doesn't apply. Use Depth as the last guard, with a limit that fits your design, and trace when it trips.

8.3 Making creation idempotent

A plugin that creates a related record must not create it twice, for example when a user saves twice or a step is retried. Before creating:

1. Check whether the record already exists (for example, an opportunity whose `originatingleadid` is this lead).
2. For a hard guarantee under concurrency, give the child table an **alternate key** and create with that key, so a second insert fails instead of duplicating.
3. Only act on a real transition: compare the pre image with Target ("rating changed from Warm to Hot"), not just the current value.

Chapter 12 applies all three.

Lab 5 — Stop a recursive update

Level: Intermediate · **Time:** 30 minutes · *This lab uses new code that isn't part of the registered D365.Plugins assembly yet. Use a test environment.*

Requirement: whenever an account's name changes, write "Name reviewed on <date>" to its Description.

Step 1 — The looping version. Add this class, build, update the assembly and register it on account **Update, PostOperation, synchronous, no filtering attributes**:

```
public class AccountReviewStampPlugin : PluginBase
{
    protected override void ExecutePlugin(LocalPluginContext localContext)
    {
        var context = localContext.PluginExecutionContext;
        var update = new Entity("account", context.PrimaryEntityId);
        update["description"] = "Name reviewed on " +
DateTime.UtcNow.ToString("yyyy-MM-dd");
        localContext.OrganizationService.Update(update);   // fires this step
again
    }
}
```

Rename an account. **Expected:** the save fails with an infinite-loop error, and the trace log shows the plugin entering at Depth 1, 2, 3 and upward.

Step 2 — The fixed version. Change the step to **PreOperation**, filter it on **name**, and write to Target:

```
public class AccountReviewStampPlugin : PluginBase
{
    protected override void ExecutePlugin(LocalPluginContext localContext)
    {
        var target = localContext.GetTargetEntity();
        if (target == null || !target.Contains("name")) return;     // guard
1 backs up the filter

        var stamp = "Name reviewed on " + DateTime.UtcNow.ToString("yyyy-MM-
dd");
        if (target.GetAttributeValue<string>("description") == stamp) return;
// guard 2

        target["description"] = stamp;                              // guard
3: no Update call
    }
}
```

Rename the account again. **Expected:** it saves once; Description shows the stamp; the trace log shows one run at Depth 1.

Step 3 — Clean up. Unregister the step when you're done.

Common errors

Symptom	Cause	Fix
Save fails with an infinite-loop error	Plugin updates the record that triggers it	Write in PreOperation, or filter and compare before Update
Rule sometimes doesn't apply	A `Depth > 1` guard skipped a legitimate nested call	Remove the blanket check; use filtering and value checks
Two related records created	No existence check or key	Check first, and add an alternate key

Knowledge check

1. Which guard removes the extra trigger completely?
2. Why might a `Depth > 1` check stop a rule from running when a flow updates the record?
3. What protects against two creates at the same moment?

Answers: 1. Writing in PreOperation (no Update call). 2. The flow's update arrives as a nested call with Depth 2. 3. An alternate key on the child table.

Summary

Loops and duplicates come from plugins that don't check whether their work is done. Filter Update steps, compare before writing, prefer PreOperation, and keep Depth as a last guard. Make record creation idempotent with existence checks and alternate keys.

Chapter 9 — Errors, transactions and asynchronous plugins

What happens when a plugin fails depends on two choices you make at registration: the stage and the mode. This chapter shows how errors flow, what gets rolled back, and when to move work to an asynchronous step.

In this chapter you will learn to:

- handle errors so users get clear messages and you get useful traces
- predict what a failure rolls back
- choose between synchronous and asynchronous execution
- monitor asynchronous steps through System Jobs

9.1 How D365.Plugins handles errors

`PluginBase.Execute` wraps every plugin in the same pattern:

```
try
{
    ExecutePlugin(localContext);
}
catch (InvalidPluginExecutionException)
{
    throw;   // business errors already carry a message for the user
}
catch (FaultException<OrganizationServiceFault> ex)
{
    localContext.Trace("OrganizationServiceFault in {0}: {1}", pluginName,
ex.ToString());
    throw new InvalidPluginExecutionException(
        string.Format("{0} could not complete the operation: {1}",
pluginName, ex.Detail.Message), ex);
}
catch (Exception ex)
{
    localContext.Trace("Unexpected error in {0}: {1}", pluginName,
ex.ToString());
    throw new InvalidPluginExecutionException(
        string.Format("An unexpected error occurred in {0}. Check the Plug-in
Trace Log for details.", pluginName), ex);
}
```

The full exception goes to the trace log; the user sees a short, safe message. Never put stack traces, connection strings or personal data in the message shown to users.

9.2 What a failure rolls back

Where the exception happens	What's rolled back	What the user sees
PreValidation, sync	The request is cancelled before the write	Your message
PreOperation or PostOperation, sync	Everything in the transaction: the record, related records, other plugins' changes	Your message
PostOperation, async	Nothing; the save already committed	Nothing; the System Job shows Failed

The trace log entry stays even when the transaction rolls back, so you can always see what happened.

9.3 Synchronous or asynchronous?

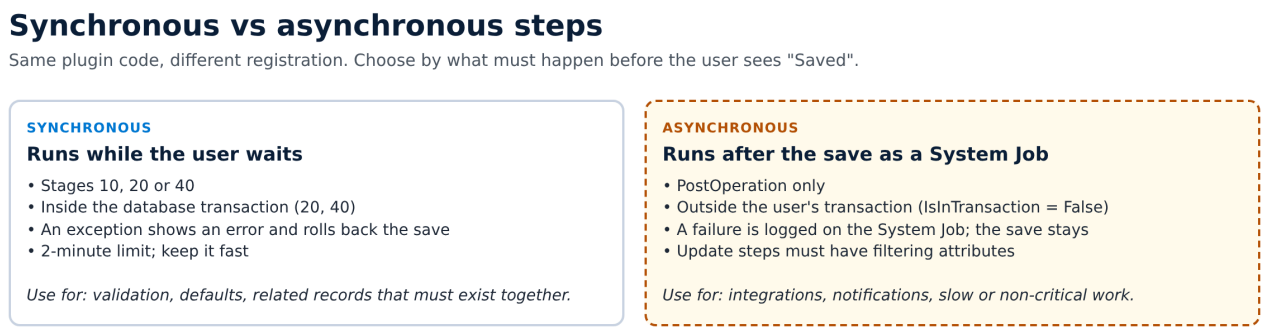


Figure 9.1 — Same plugin code, different registration.

Partial-processing risk. An async step that calls two external systems can succeed on the first and fail on the second. Design it so a retry is safe (idempotent): check whether the work was already done, or pass a unique request ID the other system can de-duplicate.

Retries. For transient errors calling external services, retry a small number of times with a short delay inside the 2-minute limit, then fail with a clear trace. Long-running or retry-heavy integrations belong in Azure Functions or Power Automate.

Lab 6 — Synchronous vs asynchronous side by side

Level: Intermediate · **Time:** 25 minutes · **Uses:** `ContextTracePlugin`.

1. Lab 1 already registered ContextTracePlugin as **async** PostOperation on account Update (filtered on name, accountnumber).
2. Register a second step for the same plugin: account **Update, PostOperation, synchronous**, filtered on `name`.
3. With tracing set to All, rename an account.
4. Open the two newest ContextTracePlugin trace log rows and compare.
5. Open **Advanced settings → System Jobs** and find the job named `D365.Plugins.Diagnostics.ContextTracePlugin: Update of account`.

Expected results. The async row below is real output from the trial; the sync row is what the same plugin reports when registered synchronously.

	Synchronous step	Asynchronous step (trial)
Mode	0	1
IsInTransaction	True	False
When it runs	Before the user sees "Saved"	A few seconds later
Where to see the result	Trace log	Trace log and a System Job with status Succeeded

6. Unregister the synchronous step when you're done.

Common errors

Symptom	Cause	Fix
Async plugin "does nothing"	System Job failed or is waiting	Check System Jobs for its status and message
Users see technical errors	Exception text passed straight through	Trace the details; show a short message
Save is slow	Slow external call in a sync step	Move it to an async step or Azure Function

Knowledge check

1. A sync PostOperation plugin creates a task and then throws. Is the task saved?
2. Where do you see that an async plugin failed?
3. Why should an async integration step be idempotent?

Answers: 1. No, the transaction rolls back. 2. In System Jobs (status Failed) and the trace log. 3. It may be retried or partly completed, so running it twice must be safe.

Summary

Throw InvalidPluginExecutionException with a clear message, trace the details, and remember that synchronous failures roll back the whole transaction. Use async steps for work that can wait, make them safe to retry, and watch them in System Jobs.

Chapter 10 — Debugging and testing

Plugins run on Microsoft's servers, so you can't just press F5. You have three tools: the trace log to see what happened, the Plug-in Profiler to replay a real run in Visual Studio, and unit tests to check logic before you deploy.

In this chapter you will learn to:

- turn on and read the Plug-in Trace Log
- capture and replay a run with the Plug-in Profiler
- write unit tests for a plugin with MSTest and Moq

10.1 The Plug-in Trace Log

Setting	Records	Use
Off	Nothing	Production default
Exception	Runs that threw an exception	Production troubleshooting
All	Every run	Development and testing only; uses storage

Change it in **Settings (gear) → Advanced settings → Settings → Administration → System Settings → Customization → Enable logging to plug-in trace log**. Read rows in **Settings → Plug-in Trace Log**, or query them with the Web API:

```
GET [org]/api/data/v9.2/plugintracelogs?
$select=typename,messageblock,createdon
    &$filter=startswith(typename,'D365.Plugins')&$orderby=createdon
desc&$top=10
```

That's exactly how the results in this book were collected. Each row has the plugin type, message, mode, depth, your trace lines and any exception.

Trace well: write one line on entry with the message, stage, depth and correlation ID (PluginBase does this), one line for each decision, and the full exception on failure. Don't trace passwords, tokens or personal data.

Correlation ID: every plugin that runs for the same request shares `context.CorrelationId`, so you can follow one save across several plugins.

10.2 The Plug-in Profiler

The profiler records the execution context of a real run so you can replay it in Visual Studio with breakpoints. It's part of Microsoft's Plug-in Registration Tool:

1. Build in **Debug** and register that build.
2. In the Plug-in Registration Tool, select **Install Profiler**.
3. Select the step and choose **Start Profiling**.
4. Reproduce the problem in the app.
5. Choose **Debug** (or **Replay Plug-in Execution**), pick the captured profile and your assembly, then attach Visual Studio to the tool's process.
6. Step through your code. When done, **Stop Profiling** and uninstall the profiler.

The profiler steps follow Microsoft's documentation; they weren't run in this book's trial, which used XrmToolBox.

10.3 Unit tests

Keep logic testable: `Execute` only gets services and calls your method; the logic reads from the context and the services it's given. Then a test can hand it fake services.

Test project file (`tests/D365.Plugins.Tests/D365.Plugins.Tests.csproj`):

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net48</TargetFramework>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.*" />
    <PackageReference Include="MSTest.TestAdapter" Version="3.*" />
    <PackageReference Include="MSTest.TestFramework" Version="3.*" />
    <PackageReference Include="Moq" Version="4.*" />
  </ItemGroup>
  <ItemGroup>
    <ProjectReference Include="..\..\src\D365.Plugins\D365.Plugins.csproj" />
  </ItemGroup>
</Project>
```

Tests for `AccountNumberValidationPlugin` :

```
using System;
using D365.Plugins.Account;
using Microsoft.VisualStudio.TestTools.UnitTesting;
using Microsoft.Xrm.Sdk;
```

```
using Moq;

[TestClass]
public class AccountNumberValidationPluginTests
{
    private static IServiceProvider BuildProvider(string message, Entity
target)
    {
        var context = new Mock<IPluginExecutionContext>();
        context.Setup(c => c.MessageName).Returns(message);
        context.Setup(c => c.PrimaryEntityName).Returns("account");
        context.Setup(c => c.Stage).Returns(10);
        context.Setup(c => c.Depth).Returns(1);
        context.Setup(c => c.UserId).Returns(Guid.NewGuid());
        context.Setup(c => c.InputParameters).Returns(new ParameterCollection
{ { "Target", target } });

        var factory = new Mock<IOrganizationServiceFactory>();
        factory.Setup(f => f.CreateOrganizationService(It.IsAny<Guid?>()))
               .Returns(new Mock<IOrganizationService>().Object);

        var provider = new Mock<IServiceProvider>();
        provider.Setup(p =>
p.GetService(typeof(IPluginExecutionContext))).Returns(context.Object);
        provider.Setup(p =>
p.GetService(typeof(IOrganizationServiceFactory))).Returns(factory.Object);
        provider.Setup(p =>
p.GetService(typeof(ITracingService))).Returns(new Mock<ITracingService>
().Object);
        return provider.Object;
    }

    [TestMethod]
    public void Create_with_valid_number_passes()
    {
        var target = new Entity("account") { ["accountnumber"] = "ACC-00001"
};
        new AccountNumberValidationPlugin().Execute(BuildProvider("Create",
target));
    }

    [TestMethod]
    [ExpectedException(typeof(InvalidPluginExecutionException))]
    public void Create_with_invalid_number_throws()
    {
        var target = new Entity("account") { ["accountnumber"] = "12345" };
```

```
        new AccountNumberValidationPlugin().Execute(BuildProvider("Create",
target));
    }

    [TestMethod]
    public void Update_without_account_number_is_ignored()
    {
        var target = new Entity("account", Guid.NewGuid()) { ["name"] =
"Renamed" };
        new AccountNumberValidationPlugin().Execute(BuildProvider("Update",
target));
    }
}
```

Integration tests go one step further: run against a dedicated test environment, create records through the API and assert on the results, as the test tables in Chapters 4–7 do. Never run them against production.

Lab 7 — Diagnose, then test

Level: Intermediate · **Time:** 40 minutes

1. Set tracing to All. Create a contact with the same email as an existing one, so ContactDuplicateEmailPlugin blocks it.
2. Find the trace log row for `ContactDuplicateEmailPlugin`. Note the entry line and the exception message.
3. Add the test project above to the solution and run **Test → Run All Tests**. **Expected:** three passing tests.
4. Add a fourth test: an account number with spaces, `" ACC-00001 "`, should pass because the plugin trims it.
5. Set tracing back to Exception.

The test code was written for this book but hasn't been run in Visual Studio yet; fix any compile issues before relying on it.

Common errors

Symptom	Cause	Fix
No trace rows	Tracing Off, or async job not run yet	Set to All; check System Jobs

Symptom	Cause	Fix
Test throws NullReferenceException	A service the plugin uses isn't mocked	Set up every `GetService` call the plugin makes
Profiler shows no profile	Step not profiled, or problem not reproduced	Start profiling on the right step, then reproduce

Knowledge check

1. Which trace log setting suits production?
2. What links trace rows from several plugins in one save?
3. Why does a test need to mock `IOrganizationServiceFactory`?

Answers: 1. Exception. 2. The correlation ID. 3. PluginBase calls it to create the organization service.

Summary

Trace every decision, read the trace log first, replay hard problems with the profiler, and cover your rules with unit tests that use mocked services.

Chapter 11 — Deploying with solutions

In development you register plugins by hand. Everywhere else, they should arrive in a **solution**, built from source control, tested, and imported the same way each time.

In this chapter you will learn to:

- put a plugin assembly, its steps and images into a solution
- move a managed solution from DEV to TEST and PROD with the Power Platform CLI
- update an assembly safely and plan a rollback

11.1 The path from code to production

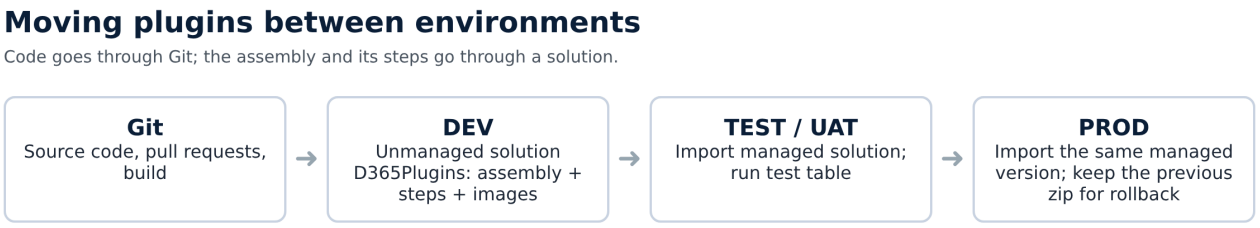


Figure 11.1 — Code moves through Git; the registered assembly and steps move through a solution.

Term	Meaning
Unmanaged solution	The editable version in DEV, where you build
Managed solution	A locked package you import into TEST, UAT and PROD; it can be cleanly removed or upgraded
Publisher	Owns the solution and sets the prefix for new components
Environment variable	A per-environment setting (such as an API URL) that travels with the solution, with a different value in each environment

11.2 What goes in the solution

Component	How to add it
Plug-in assembly	Registration tool → select assembly → **Add To Solution**; or in make.powerapps.com: Solutions → your solution → **Add existing → More → Developer**
Plug-in steps and their images	Same, choosing each step (images travel with their step)
Environment variables	Solutions → **New → More → Environment variable**

Secure step configuration does **not** travel with a solution; set it in each environment after import.

11.3 Export and import with the Power Platform CLI

```
# Connect to DEV
pac auth create --environment https://yourdev.crm8.dynamics.com

# Unmanaged export -> unpack into Git (the source of truth for the solution)
pac solution export --name D365Plugins --path .\out\D365Plugins.zip
pac solution unpack --zipfile .\out\D365Plugins.zip --folder
.\solution\D365Plugins

# Managed export -> the file you release
pac solution export --name D365Plugins --path .\out\D365Plugins_managed.zip -
-managed

# Connect to TEST and import the managed file
pac auth create --environment https://yourtest.crm8.dynamics.com
pac solution import --path .\out\D365Plugins_managed.zip
```

In a pipeline (Azure DevOps or GitHub Actions), the same steps run on every merge: build the assembly, run unit tests, pack the solution, import to TEST, run the test tables.

11.4 Updating an assembly

1. Change the code and raise the **build or revision** number (1.0.0.0 → 1.0.1.0). Keep major.minor the same for an in-place update.
2. In DEV, select the assembly in the registration tool and choose **Update**, pointing to the new DLL. Steps stay registered.

3. Export a new solution version and import it downstream.

Rollback: keep the previous managed solution zip for each release. If a release fails, re-import the previous version, or disable the faulty step while you fix it.

Removing a step safely: disable it first and watch for a release cycle, then unregister it in DEV and ship the solution so it's removed downstream too.

Lab 8 — Move D365.Plugins to another environment

Level: Intermediate · **Time:** 45 minutes · **Needs:** a second environment (a second trial or a developer environment).

1. In make.powerapps.com, create a publisher and a solution named **D365Plugins**.
2. Add the D365.Plugins assembly and all nine steps (Section 11.2).
3. Export it as **managed** with `pac solution export` (Section 11.3).
4. Import it into the second environment with `pac solution import`.
5. In the second environment, set the CasePriorityPlugin keywords and run the test table from Chapter 4.

Acceptance: all nine steps appear in the second environment; the tests give the same results as in DEV.

This lab wasn't run in the book's single trial environment; it follows Microsoft's documented CLI commands. Check `pac solution export --help` for the options in your CLI version.

Release checklist

- [ ] Unit tests pass; test table passes in TEST
- [ ] Assembly version raised (build or revision)
- [ ] Solution exported as managed and stored with the release
- [ ] Previous release zip kept for rollback
- [ ] Secure configuration set in the target environment
- [ ] Trace log set to Exception in PROD

Knowledge check

1. Why import a managed solution into PROD instead of registering by hand?
2. Which step setting doesn't travel with a solution?
3. What version change keeps an in-place assembly update?

Answers: 1. It's repeatable, versioned and can be cleanly upgraded or removed. 2. Secure configuration. 3. Changing only the build or revision number.

Summary

Build in DEV, keep code and unpacked solutions in Git, and ship managed solutions downstream with the CLI or a pipeline. Version every release and keep the previous one for rollback.

Chapter 12 — Capstone: Lead to Opportunity

This capstone puts the whole book together in one feature: when a lead's rating becomes **Hot**, create one opportunity for it, exactly once, with a clear trace of what happened.

You will use: a filtered Update step (Ch 2, 8), a pre image to detect a real change (Ch 5), PostOperation record creation (Ch 7), an idempotency check (Ch 8), error handling (Ch 9), unit tests (Ch 10) and a solution (Ch 11).

12.1 Requirement

#	Rule
R1	When a lead's Rating changes to Hot, create an opportunity
R2	Name the opportunity after the lead's Topic; link it to the lead and, if set, the lead's account
R3	Never create a second opportunity for the same lead
R4	Do nothing when other lead columns change, or when Rating was already Hot
R5	If the opportunity can't be created, the rating change must not be saved

R5 means the step is **synchronous PostOperation**: the lead and opportunity are saved together or not at all.

12.2 Design

Decision	Choice	Why
Message and table	Update of `lead`	Rating changes are updates
Stage and mode	PostOperation, synchronous	Need the saved lead; same transaction (R5)
Filtering attributes	`leadqualitycode`	Runs only when Rating changes (R4)

Decision	Choice	Why
Pre image	`PreImage`: leadqualitycode, subject, parentaccountid	Compare old and new rating; read Topic without a retrieve
Idempotency	Query for an opportunity whose `originatingleadid` is this lead	R3

Lead Rating (`leadqualitycode`) values: 1 = Hot, 2 = Warm, 3 = Cold.

12.3 The plugin

```
using D365.Plugins.Common;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Query;

namespace D365.Plugins.Lead
{
    public class LeadHotToOpportunityPlugin : PluginBase
    {
        private const int RatingHot = 1;

        protected override void ExecutePlugin(LocalPluginContext
localContext)
        {
            var context = localContext.PluginExecutionContext;
            if (!localContext.IsStep("lead", "Update",
PipelineStage.PostOperation)) return;

            var target = localContext.GetTargetEntity();
            if (target == null || !target.Contains("leadqualitycode"))
return;

            var newRating = target.GetAttributeValue<OptionSetValue>
("leadqualitycode");
            var pre = localContext.GetPreImage("PreImage");
            var oldRating = pre != null ?
pre.GetAttributeValue<OptionSetValue>("leadqualitycode") : null;

            // R1 and R4: act only on a real change to Hot.
            if (newRating == null || newRating.Value != RatingHot) return;
            if (oldRating != null && oldRating.Value == RatingHot) return;

            var leadId = context.PrimaryEntityId;
```

```
            // R3: one opportunity per lead.
            var existing = new QueryExpression("opportunity") { ColumnSet =
new ColumnSet(false), TopCount = 1 };
            existing.Criteria.AddCondition("originatingleadid",
ConditionOperator.Equal, leadId);
            if
(localContext.OrganizationService.RetrieveMultiple(existing).Entities.Count >
0)
            {
                localContext.Trace("Opportunity already exists for lead {0};
nothing to do.", leadId);
                return;
            }

            // R2: build the opportunity from the pre image.
            var topic = pre != null ? pre.GetAttributeValue<string>
("subject") : null;
            var opportunity = new Entity("opportunity");
            opportunity["name"] = string.IsNullOrWhiteSpace(topic) ?
"Opportunity from hot lead" : topic;
            opportunity["originatingleadid"] = new EntityReference("lead",
leadId);

            var account = pre != null ?
pre.GetAttributeValue<EntityReference>("parentaccountid") : null;
            if (account != null) opportunity["parentaccountid"] = account;

            var id = localContext.OrganizationService.Create(opportunity);
            localContext.Trace("Created opportunity {0} for lead {1}.", id,
leadId);
        }
    }
}
```

Concurrency: two users setting Rating to Hot at the same instant could both pass the existence check. For a hard guarantee, add an alternate key on the opportunity's `originatingleadid` column so the second create fails and its save rolls back.

12.4 Registration

Setting	Value
Message / table	Update / lead
Stage / mode	PostOperation / Synchronous

Setting	Value
Filtering attributes	leadqualitycode
Image	Pre Image, name and alias `PreImage`, columns leadqualitycode, subject, parentaccountid

12.5 Test plan

#	Action	Expected
T1	Lead Warm → Hot	One opportunity, named after the Topic, linked to the lead
T2	Same lead: Hot → Warm → Hot	Still one opportunity; trace says it already exists
T3	Change the lead's phone number	Plugin doesn't run (filtering)
T4	Lead created as Hot	No opportunity (rule is about changes; decide with the business whether Create needs a step too)
T5	Make opportunity creation fail (for example, remove the user's create permission on opportunity)	Rating change rejected with a clear error; lead stays Warm

Add unit tests for T1, T2 and T4 using the pattern in Chapter 10: mock `RetrieveMultiple` to return zero or one opportunity and assert whether `Create` was called.

Capstone deliverables

- [ ] `LeadHotToOpportunityPlugin.cs` in `src/D365.Plugins/Lead/`
- [ ] Unit tests for T1, T2, T4 passing
- [ ] Step and image registered as in 12.4
- [ ] T1–T5 run in a test environment, with screenshots
- [ ] Assembly, step and image added to the D365Plugins solution and exported as managed

This capstone's code was written for the book and hasn't been built or run in the trial yet. Lead is part of the Sales schema; check that your trial has the lead table and an app that shows it.

Summary

A production-ready plugin acts only on real changes, does its work once, keeps related writes in one transaction, explains itself in the trace log, is covered by tests, and ships in a solution.